

Semester Project

A Visual Resolution for Syntactic Ambiguity

Autumn Term 2024

Declaration of Originality

I hereby declare that the written work I have submitted entitled

Unraveling Annotator Disagreement: Ambiguity and Subjectivity

is original work which I alone have authored and which is written in my own words.

Author(s)

Yiwei

Liu

Student supervisor(s)

Robin

Chan

Supervising lecturer

Frédéric
Mennatallah

Kaplan
El-Assady

With the signature, I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on 'Citation etiquette' (<https://www.ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/plagiarism-citationetiquette.pdf>). The citation conventions usual to the discipline in question here have been respected.

The above written work may be tested electronically for plagiarism.

Place and date

Signature

Contents

Preface	v
Abstract	vii
Symbols	ix
1 Introduction	1
2 Background	3
2.1 Linguistic Background of Ambiguity	3
2.1.1 Definition of Ambiguity	3
2.1.2 Syntactic Ambiguity	3
2.2 Constituency Parsing	5
3 Related Work	7
3.1 Ambiguity Challenges in NLP Systems	7
3.2 Interactive Systems and Visualization	8
4 Method	10
4.1 Dataset	10
4.2 Prompt-Based Classification	11
4.2.1 Prompt Design	11
4.2.2 Model Selection	12
4.2.3 Evaluation Metrics	12
4.3 K-best Constituency Parsing for Syntactic Ambiguity Detection	12
4.3.1 Parsing Model	12
4.3.2 Ambiguity Detection from Parse Score Distributions	13
5 Results	15
5.1 Prompt-Based Classification Results	15
5.2 K-best Constituency Parsing for Syntactic Ambiguity Detection Results	16
5.2.1 Manual Verification of Parse Trees	16
5.2.2 Statistical Analysis of Parse Score Distributions	18
5.2.3 Performance Across Ambiguity Types	20
6 A Visual Resolution for Syntactic Ambiguity	21
6.1 Design Goals and Motivation	21
6.2 Visualization Components	21
6.3 Technical Implementation	24

7	Discussion	25
7.1	Effectiveness of Different Approaches	25
7.2	Model Bias and Role of Ambiguity in Language	25
7.3	Future Directions	26
8	Conclusion	27
	Bibliography	33
A	Prompts and Model Descriptions	35
A.1	The Prompt Template	35

Preface

This work represents my semester project completed during the 2024 autumn semester. I would like to express my sincere gratitude to my supervisor, Robin, for his patient guidance and invaluable support throughout this project. His insightful feedback and encouragement have been instrumental in shaping this work. Also, I am deeply grateful to Professor El-Assady for her valuable guidance and suggestions. Their expertise and dedication have not only helped improve the quality of this work but also enhanced my understanding of how to conduct rigorous research.

Abstract

This thesis presents a novel approach to detecting and visualizing syntactic ambiguity in natural language, addressing a critical challenge in human-AI interaction. We introduce a method based on k-best constituency parsing that analyzes parse score distributions to identify ambiguous sentences, validated through statistical measures including standard deviation, entropy, and distribution divergence. Our experimental results demonstrate the effectiveness of this approach, particularly for prepositional phrase attachment ambiguity, while revealing varying performance across different syntactic ambiguity types.

We also present an interactive visualization system with four complementary visualization components: visual markers, contextual explanations, disambiguation examples, and parse tree visualizations, making these complex linguistic structures accessible to non-expert users. This work contributes to both the technical understanding of ambiguity detection and the practical aspects of human-AI communication, providing a foundation for more effective ambiguity handling in natural language processing systems.

Symbols

Acronyms and Abbreviations

ETH	Eidgenössische Technische Hochschule
LLM	Large Language Models
HCI	Human-Computer Interaction

Chapter 1

Introduction

Ambiguity is an inherent feature of natural language. While ambiguities may lead to different interpretations, they can allow for greater efficiency in communication [1]. Different types of ambiguity may arise, among which is syntactic ambiguity. As defined in Li et al. [2], syntactic ambiguity occurs when multiple grammatical structures are possible for a sequence of words. Humans have evolved effective mechanisms to handle ambiguity through interaction and clarification questions [3], yet these processes often break down in human-machine contexts, creating significant challenges across different aspects of Natural Language Processing (NLP) systems. Clear and unambiguous communication thus becomes crucial for effective Human-Computer Interaction (HCI), yet recent studies have highlighted that current Large Language Models (LLMs) struggle to handle ambiguity as effectively as humans do [4, 5]. For instance, in code generation tasks, ambiguity in user instructions often leads to misalignment between users’ intentions and LLMs’ interpretations [6].

This human-machine gap in handling ambiguity also manifests in the creation of NLP training data. In annotation tasks, different interpretations of ambiguous text often lead to what Pavlick and Kwiatkowski [7] define as inherent disagreements, which are persistent differences in understanding that cannot be resolved even with additional context and cannot be dismissed as mere annotation “noise”. These inherent disagreements manifest when annotators encounter syntactically ambiguous sentences, where different parsing choices lead annotators to derive distinct interpretations from the same text. The severity of this issue is evident in recent studies on dataset creation for language model alignment [8], where up to 50% of annotations were excluded due to low inter-annotator agreement.

These challenges caused by the human-machine gap in handling ambiguity underscore the critical need for proactive interfaces that can identify and visualize potential ambiguities. While humans can often resolve ambiguity through interaction and clarification, this is not always possible—particularly with written text where additional context cannot be queried. This challenge affects both human readers and computer systems attempting to process ambiguous text, though in different ways. Such interfaces would enable users to recognize and address potential ambiguity before it impacts comprehension, whether by human readers or computational systems.

In this work, we mainly focus on syntactic ambiguity and explore effective methods for its detection and visualization. To address this need, an interactive visualization approach is proposed in this project that not only detects syntactic ambiguity, but also makes different possible interpretations explicit and accessible to users. The key contributions include:

- A novel method for detecting syntactic ambiguity using K-best Constituency

parsing, which enables the identification of syntactic ambiguity

- An interactive visualization system that presents syntactic structures through multiple coordinated views, enabling users to explore and compare different interpretations intuitively

The source code and interactive visualization system developed in this project are available at: <https://github.com/dh-epfl-students/dhlab-visual-resolution-syntactic-ambiguity>.

This approach goes beyond mere detection by providing users with visual tools to understand and resolve potential ambiguities. The focus is on syntactic ambiguity and this visualization-based solution has broader implications in various scenarios, like writing assistance where clear and unambiguous expression is crucial.

Chapter 2

Background

2.1 Linguistic Background of Ambiguity

2.1.1 Definition of Ambiguity

Ambiguity is a pervasive phenomenon of natural language whereby words have multiple meanings and syntactic categories when out of context. While traditionally viewed as a limitation or flaw in communication, research has shown that ambiguity actually serves as an efficient feature of language systems by allowing linguistic units to be re-used [1]. Recent work by Li et al. [2] proposes a comprehensive taxonomy of ambiguity types in English, particularly relevant to modern NLP contexts. Their framework identifies eleven distinct types of ambiguity: lexical ambiguity, syntactic ambiguity, scopal ambiguity, elliptical ambiguity, collective/distributive ambiguity, implicative ambiguity, presuppositional ambiguity, idiomatic ambiguity, coreferential ambiguity, generic/nongeneric ambiguity, and type/token ambiguity.

2.1.2 Syntactic Ambiguity

Among these various types of ambiguity, this project primarily focuses on syntactic ambiguity.

Examples and Analysis

Syntactic ambiguity can be better understood through concrete examples. A clear illustration can be found in the SNLI (Stanford Natural Language Inference) dataset. Consider this premise from Chan et al. [9]: “A fire hydrant that has been painted to look like it has a face,” paired with the hypothesis: “there is a painting of a fire hydrant.” The syntactic ambiguity in this sentence stems from the relationship between two core components:

- Main subject: “A fire hydrant”
- Modifying clause: “that has been painted to look like it has a face”

This structural relationship leads to two distinct interpretations:

- Interpretation 1: A real fire hydrant has been painted to make it look like it has a face. Here, the modifying clause directly describes a physical fire hydrant, where “painted” refers to the act of painting the actual hydrant.

- Interpretation 2: This is a painting of a fire hydrant that looks like it has a face. In this interpretation, the entire phrase describes an artwork, where “painted” refers to the creation of the artwork itself.

Such structural ambiguity poses significant challenges in NLP tasks, particularly in scenarios requiring human annotation. For instance, in NLI tasks, the relationship between premise and hypothesis critically depends on which interpretation is adopted, leading to different logical relationships and classifications. As noted by Pavlick and Kwiatkowski [7], this ambiguity directly impacts annotation quality through annotator disagreement. Further research by Liu et al. [4] found that when sentences contain inherent ambiguity, individual annotators recognize this uncertainty, often resulting in systematic disagreements in the annotation process. These findings highlight the importance of addressing syntactic ambiguity in NLP systems.

Types of Syntactic Ambiguity

Syntactic ambiguity can arise from various linguistic structures, with different types emerging from the specific properties of words and phrases in a sentence. As Nadh and Huyck [10] points out, the range of syntactic ambiguities is broad, encompassing phenomena such as conjunctive ambiguity and ambiguity between relative clauses and main verbs.

In this project, we focus particularly on two types of syntactic ambiguity: prepositional phrase attachment and compound noun attachment. These types are particularly interesting due to their frequent occurrence in natural language and their significant impact on sentence interpretation. Detailed definitions and an example of each type are described below.

Prepositional Phrase-attachment ambiguity Among various types of syntactic ambiguity, prepositional phrase (PP) attachment ambiguity represents one of the most extensively studied phenomena in computational linguistics. This type of ambiguity occurs when a prepositional phrase can be grammatically attached to different parts of a sentence. A classic example is the sentence “I saw the girl with the telescope” [10], which yields two distinct interpretations:

- Verb Phrase (VP) attachment: The telescope serves as an instrument for seeing (I used the telescope to see the girl)
- Noun Phrase (NP) attachment: The telescope is associated with the girl (I saw the girl who had a telescope)

The ambiguity arises because the prepositional phrase “with the telescope” can logically modify either the verb “saw” or the noun phrase “the girl” leading to fundamentally different semantic interpretations of the same sentence.

Compound noun attachment ambiguity One common type of syntactic ambiguity occurs in noun compounds, which are linguistic structures where multiple nouns are combined to function as a single noun unit. While English noun compounds typically follow a right-headed structure, their recursive nature can lead to structural ambiguities when both the modifier and the head can be compounds themselves [11]. The syntactic structure of compound nouns is formed through recursive combinations of nouns ($N \rightarrow NN$). When a compound contains more than two nouns, this will lead to ambiguity in syntactic structure[12].

To illustrate this type of ambiguity, consider this example: the phrase “superfluous hair remover” [13, 2].

- (superfluous)(hair remover): A hair remover that is unnecessary
- (superfluous hair)(remover): A device that removes unwanted hair

The ambiguity arises because the adjective “superfluous” can modify either the entire compound noun “hair remover” or just the noun “hair”, leading to distinctly different meanings.

Prepositional phrase (PP) attachment and compound noun attachment represent common forms of syntactic ambiguity prevalent in everyday language, presenting significant challenges in both HCI and NLP. Addressing these ambiguities and providing users with appropriate guidance for understanding them is crucial for improving the effectiveness of language-based interactions between humans and machines.

2.2 Constituency Parsing

Constituency parsing, which aims to generate a structured syntactic parse tree for a given sentence, is a fundamental task in natural language understanding. Traditionally, this task relied on Context-Free Grammar (CFG), a formal system that defines rules for modeling constituent structure in natural language. CFG consists of a set of production rules that specify how language symbols can be grouped and ordered, along with a lexicon of words and symbols [14].

In recent years, neural network approaches have revolutionized constituency parsing. Neural parsers have demonstrated superior performance compared to conventional statistical grammar-based approaches, primarily due to their ability to capture long-range contextual information effectively. Particularly, recent studies have shown significant improvements in chart-based parsers using neural architectures, especially with the integration of recurrent models like Bi-LSTM [15, 16].

The parsing process typically employs the Cocke-Younger-Kasami (CYK) algorithm [17], a dynamic programming approach that determines whether a given string belongs to a context-free grammar. This algorithm works with grammar rules in Chomsky Normal Form and systematically builds parse trees through a bottom-up approach.

Span-based Parsing

The understanding of modern constituency parsing builds upon the span attention framework introduced by Tian et al. [15]. In this framework, constituency parsing typically employs a span-based paradigm, where a parse tree $\mathcal{T}$ for an input sentence $\mathcal{X} = x_1x_2\dots x_n$ is represented through labeled spans. Each span is defined by a triplet (i, j, l) , where i and j denote the beginning and ending positions, and $l \in \mathcal{L}$ represents the constituent label from the label set $\mathcal{L}$. In modern neural architectures, spans are typically encoded using contextual representations. For a span (i, j) , its representation $r_{i,j}$ is computed from the difference of contextual embeddings at positions i and j . These span representations are then processed through neural layers to compute scores $s(i, j, l)$ for each possible constituent label:

$$s(i, j, l) = f_{\theta}(r_{i,j}) \quad (2.1)$$

where f_{θ} represents a neural scoring function with parameters θ . The final parse tree is constructed by selecting the combination of spans that maximizes the overall tree score while maintaining valid tree constraints. This span-based approach, combined with the CYK algorithm’s dynamic programming strategy, enables efficient parsing while leveraging the power of neural representations. The CYK algorithm builds a parsing table $P[n, n, r]$, where n is the sentence length and r is the number of possible constituent labels.

For a sentence with spans scored by function $s(i, j, l)$, the best score $s^*(i, j)$ for span (i, j) is computed recursively:

$$s^*(i, j) = \max_{l \in \mathcal{L}} s(i, j, l) + \max_{i < k < j} [s^*(i, k) + s^*(k, j)] \quad (2.2)$$

The dynamic programming process follows three main steps:

1. **Initialization:** For adjacent words ($j = i + 1$), compute the best score using only the span labels:

$$s^*(i, i + 1) = \max_{l \in \mathcal{L}} s(i, i + 1, l) \quad (2.3)$$

2. **Bottom-up Construction:** For increasing span lengths, compute optimal scores by considering all possible split points k and combining the scores of smaller spans.
3. **Tree Recovery:** Once $s^*(1, n)$ is computed for the full sentence, use backpointers to reconstruct the optimal parse tree by tracing the decisions made during the bottom-up process.

Chapter 3

Related Work

3.1 Ambiguity Challenges in NLP Systems

Syntactic ambiguity, as introduced in Section 2.1.2, has been a fundamental challenge in natural language processing since its inception. As noted by Nadh and Huyck [10], the main difficulty in natural language parsing lies in the frequent ambiguity of syntactic structures, where semantic understanding is often necessary for resolution. Early approaches to syntactic ambiguity resolution focused on using various semantic-based methods. Foundational experiments conducted on PP-attachment by Ferstl [18], revealing that sentence bias and contextual factors, rather than pure syntactic structure, determine processing decisions. For compound noun ambiguity, Buckeridge and Sutcliffe [19] proposed using Latent Semantic Indexing to provide domain-independent semantic knowledge, addressing the limitations of previous statistical methods that relied heavily on pre-existing unambiguous examples. Building on semantic approaches, a system developed by Nadh and Huyck [10] utilizing WordNet’s semantic hierarchies for PP-attachment resolution, achieving 90.53% accuracy on the Penn Treebank dataset. This was further advanced by Olteanu and Moldovan [20], who incorporated broader contextual information and web-derived knowledge, improving accuracy to over 93%. While these traditional approaches demonstrated the importance of semantic information and context in disambiguation, they typically required extensive pre-existing knowledge or context, making them impractical for real-time ambiguity detection in isolated sentences. This becomes particularly challenging in NLP tasks, where precise interpretation is crucial for downstream applications.

With the advent of LLMs, research focus has shifted from traditional rule-based and statistical approaches to exploring how these powerful models handle different types of ambiguity. Beyond syntactic ambiguity, different types of ambiguity present distinct challenges for LLMs, creating a significant gap in understanding how models respond to various forms of ambiguity [2]. Several studies have explored this across different ambiguity types, revealing varying levels of model capability. For instance, [5] developed AMBICOREF, a diagnostic corpus for coreferential ambiguity, demonstrating that most coreference resolution models, unlike humans, fail to distinguish between ambiguous and unambiguous cases. In the domain of semantic ambiguity, Liao et al. [21] found that while LLMs show human-like patterns in Chinese text segmentation through statistical learning, their processing mechanisms fundamentally differ from human cognition. Conversely, Kamath et al. [22] reported more promising results in scope ambiguity detection, where modern LLMs achieved over 90% accuracy in aligning with human interpretations. These findings highlight the heterogeneous nature of model capabilities across different types of ambiguity,

suggesting that current LLMs’ ability to handle ambiguity remains inconsistent and potentially unreliable. This variability in performance underscores the need for more robust and reliable methods for ambiguity detection and resolution in NLP systems. Recent efforts to address ambiguity in LLMs have primarily focused on two main approaches: prompting-based methods and model calibration. Prompting-based approaches, such as enabling models to ask clarifying questions [3] or generating targeted prompts [23], offer general solutions but fail to address the specific challenges posed by different types of ambiguity. While these methods have shown success in certain applications, improving code generation accuracy by up to 10% [23], they lack the specificity needed for handling distinct ambiguity types. Model calibration approaches, including fine-tuning techniques [4] and explicit alignment methods [24], show promise in addressing specific types of ambiguity but still face significant challenges, as evidenced by GPT-4’s limited 32% accuracy in generating correct disambiguations [4].

These limitations underscore the critical need for specialized approaches that can detect and handle different types of ambiguity independently. Particularly in the domain of syntactic ambiguity, where structural interpretations can significantly impact downstream applications, there remains a notable gap in reliable automatic detection methods. This gap becomes especially critical given the increasing reliance on LLMs in various NLP applications where precise interpretation is essential.

3.2 Interactive Systems and Visualization

Current LLM-based interfaces, such as ChatGPT [25], primarily rely on linear, text-based conversational structures, which are often inadequate for managing complex linguistic phenomena [26], particularly in providing real-time visual feedback for ambiguity detection and resolution. To address these limitations, recent work has explored various visualization approaches to enhance user understanding of complex linguistic information. For example, Graphologue [26] automatically converts text-based conversations into interactive node-link diagrams, allowing users to explore and understand complex dialogue structures through visual navigation. RELIC [27] takes a different approach by visualizing the self-consistency patterns across multiple LLM-generated responses, enabling users to identify potential inaccuracies through comparative analysis of semantic variations. While these systems demonstrate the value of visual elements in making complex information more accessible, they are not specifically designed for ambiguity detection.

In the domain of syntactic structure visualization, existing approaches have explored different directions, ranging from expert-oriented tools to more accessible solutions. Sánchez-Sáez et al. [28] developed an interactive tree annotation system focusing on expert linguistic analysis, while Berzak et al. [29] explored a more accessible approach using visual scenes for disambiguation. However, these existing approaches face several key limitations. Specialized tools like interactive parsing trees, while providing detailed syntactic information, remain too complex for non-expert users to interpret. More accessible visualization approaches, such as using illustrative representations to depict different possible interpretations of ambiguous sentences, have only been explored in isolated examples rather than being systematically integrated into interactive systems. Moreover, current systems typically operate retrospectively, providing feedback only after sentence completion rather than during the process.

To address these limitations, there is a clear need for a system that combines real-time detection with intuitive visual representations, enabling immediate awareness and resolution of ambiguities during the writing process. This would represent a significant advancement over the post-hoc feedback approaches currently employed

by systems like ChatGPT [25].

Chapter 4

Method

4.1 Dataset

This study utilizes a subset of the AmbiEnt dataset introduced by Liu et al. [4], focusing specifically on syntactic ambiguity examples for our parsing-based detection approach, while also including lexical and coreferential ambiguity cases for comparative analysis in prompt-based classification experiments.

The AmbiEnt dataset categorizes ambiguity into seven distinct types, as shown in Table 4.1.

Category	Description
Lexical Ambiguity	Multiple possible word senses
Syntactic Ambiguity	Multiple valid syntactic parse structures
Figurative Ambiguity	Coexistence of literal and figurative interpretations
Pragmatic Ambiguity	Distinction between literal and pragmatic meanings
Scopal Ambiguity	Multiple interpretations due to quantifier scope or modifier attachment
Coreferential Ambiguity	Unclear referential relationships
Other	Cases not falling into the above categories

Table 4.1: Definitions of Ambiguity Types

From the complete dataset distribution shown in Table 4.2, the experimental design structured the dataset usage as follows:

- Prompt-based Classification: The analysis focused on syntactic, lexical, and coreferential categories, excluding other categories due to insufficient sample sizes
- K-best Constituency Parsing for Syntactic Ambiguity Detection: The study specifically utilized the syntactic ambiguity samples to evaluate the parsing-based approach

The dataset contains two instances labeled with multiple categories (pragmatic and other, pragmatic and scopal). These cases were excluded from the analysis to avoid potential complications for the classification task arising from overlapping categories. To evaluate the effectiveness of the K-best constituency parsing approach in distinguishing ambiguous from non-ambiguous sentences, this study incorporated a group of presumptively non-ambiguous sentences. These sentences were randomly sampled from the premise statements of MultiNLI dataset [30], which is originally

Category	Sample Count
Syntactic	9
Lexical	21
Coreferential	3
Pragmatic	45
Scopal	7
Figurative	2
Other	14

Table 4.2: Distribution of Ambiguity Types in the Dataset

composed of premise-hypothesis pairs. Fifty samples were extracted from the training set to match the size of the syntactic ambiguity samples in the AmbiEnt dataset. The MultiNLI dataset was selected as the source of non-ambiguous sentences for several key reasons: its premises typically consist of straightforward statements with clear interpretations, facilitating unambiguous NLI task annotations. It has been extensively validated across various NLP tasks, and it contains naturally occurring English sentences. This combination of ambiguous sentences from AmbiEnt [4] and non-ambiguous sentences from MultiNLI [30] enables a comprehensive evaluation of the parsing approach across both ambiguous and non-ambiguous cases.

4.2 Prompt-Based Classification

4.2.1 Prompt Design

Given that Large Language Models (LLMs) are prone to generating non-factual content and hallucinations [31], prompt strategy is crucial for obtaining reliable responses. This project employs three main prompting strategies to enhance the quality and reliability of model outputs.

The first strategy incorporates chain-of-thought reasoning [32], which encourages models to break down complex reasoning tasks into intermediate steps. This approach was implemented by explicitly requesting “think step-by-step” and “provide step-by-step reasoning analysis” in the prompts, allowing models to demonstrate their reasoning process rather than providing direct classifications. This strategy was applied across all selected models to evaluate its effectiveness in different model architectures and scales.

The second strategy focuses on reducing hallucinations through self-consistency checking. This approach involves a two-stage process: an initial classification followed by a challenge phase where the model is prompted to reconsider its answer through questions such as “Are you sure about this answer?”, encouraging a more thorough evaluation of specific interpretations rather than relying solely on independent generation. This method aligns with recent developments in self-verification techniques, such as SelfCheckGPT [33], which employs multiple-choice questions to evaluate response consistency. Due to the known limitations of sycophantic behavior in LLMs identified in the FlipFlop experiment [34], this strategy was exclusively tested with GPT-4 to maintain experimental reliability.

For each model, multiple prompt variants were tested within these strategic frameworks. The most effective prompts were selected based on preliminary testing results and subsequently employed in the main experiments. The complete prompt used in this study is provided in the appendix A.1. The prompt follows a “think step by step” approach, where the model is first provided with definitions of different ambiguity types, then asked to determine whether a given sentence contains ambiguity, and if

so, to classify it according to the provided definitions. The “flip-flop” component is integrated into this base structure to encourage more thorough consideration of the initial response. The complete set of prompts used in this study is provided in the appendix A.1.

These prompt-based experiments were conducted to establish a baseline for LLMs’ ambiguity detection capabilities, providing context for the subsequent development of the more specialized method of K-best parse based approach.

4.2.2 Model Selection

This project evaluated several widely-adopted and state-of-the-art language models, including both closed-source and open-source options. Specifically, this project evaluated three prominent model families that have demonstrated outstanding complex reasoning capabilities across various NLP tasks: GPT-4 [35] (accessed through ChatGPT interface), Llama-3 [36], and Phi-3 [37]. While GPT-4’s parameters are not publicly disclosed, Llama-3 was tested in two variants: a smaller version with 8 billion parameters (8B) and a larger version with 405 billion parameters (405B). Phi-3, notably smaller, comprises approximately 128,000 parameters (128K). Here, “B” denotes billion (10^9) and “K” indicates thousand (10^3).

This selection of models spans different architectures, parameter sizes, and accessibility levels, enabling a comprehensive evaluation of ambiguity detection capabilities across various model scales and types. The inclusion of both proprietary (GPT-4) and open-source models (Llama-2, Phi-3) also allows for comparison between different development approaches and their impact on ambiguity handling.

4.2.3 Evaluation Metrics

The performance of the prompt-based classification was evaluated using accuracy, calculated as the ratio of correct classifications to the total number of samples:

$$\text{Accuracy} = \frac{\text{Number of Correct Classifications}}{\text{Total Number of Samples}} \quad (4.1)$$

This metric directly measures the models’ ability to correctly identify ambiguity types.

4.3 K-best Constituency Parsing for Syntactic Ambiguity Detection

4.3.1 Parsing Model

As defined in Li et al. [2], syntactic ambiguity occurs when multiple grammatical structures are possible for a sequence of words, manifesting as multiple plausible syntactic structures with comparable probabilities. While traditional constituency parsing typically outputs the single most probable parse tree [16, 15], the parsing model is modified to generate K-best outputs, enabling analysis of the distribution of parsing trees and their associated scores.

The neural chart-based constituency parsing model from Tian et al. [15], which achieved the highest F1 score of 96.40 on the Penn Treebank leaderboard [?], is used as the foundation. To enable syntactic ambiguity detection, the original parsing algorithm is modified to generate multiple high-scoring parse trees instead of a single best parse:

Span Score Instead of maintaining only the highest scoring option for each span, the model's data structures are extended to preserve the k -best candidates for each span (i, j) . This modification enables the tracking of multiple plausible syntactic interpretations at each level of the parsing process.

Dynamic Programming Algorithm Adaptation The CYK algorithm is adapted to maintain k -best combinations throughout the bottom-up construction process:

- For each span (i, j) , all possible split points s are considered
- For each split point, the k -best results from the left span (i, s) and the right span (s, j) are combined
- k^2 candidate combinations are generated
- The k -best combinations based on their scores are selected and stored

This modified parsing approach produces k parse trees for each input sentence, along with their associated tree scores, providing the foundation for the ambiguity detection method.

4.3.2 Ambiguity Detection from Parse Score Distributions

The approach to detecting syntactic ambiguity relies on analyzing the distribution patterns of parse scores. It is hypothesized that syntactically ambiguous sentences will produce multiple parse trees with comparably high scores, each corresponding to a different valid interpretation, while unambiguous sentences will typically yield one dominant high-scoring parse tree with notably lower scores for alternatives.

To validate this hypothesis, a manual verification process is first conducted. The trees with the highest scores are examined to verify whether they correspond to the different valid interpretations documented in the ambiguous sentence dataset, rather than representing parsing noise or invalid structures. This step is crucial to ensure that the score-based detection method is capturing genuine syntactic ambiguity rather than model artifacts.

The analysis framework thus encompasses three types of parse score distributions:

- Distributions from ambiguous sentences (S_a)
- Distributions from their disambiguated versions (S_d)
- Distributions from naturally non-ambiguous sentences (S_n)

The robustness of the detection method in distinguishing between ambiguous and non-ambiguous sentences is evaluated through statistical analysis of parse score distributions. Three main statistical approaches are employed:

- **Standard Deviation Analysis:** The standard deviation of parse scores is calculated for each sentence to measure the spread of scores. The distributions of these standard deviations for ambiguous and non-ambiguous sentences are compared using both independent t-tests and Mann-Whitney U tests to assess statistical significance of their differences.
- **Entropy Analysis:** For each sentence, the entropy of its normalized score distribution is computed. This measure quantifies the uncertainty or randomness in the score distribution. The mean and standard deviation of entropy values are compared between ambiguous and non-ambiguous sentences to identify systematic differences.

- **Distribution Divergence:** The overall difference between ambiguous and non-ambiguous score distributions is measured using both Jensen-Shannon and Kullback-Leibler divergence metrics. These information-theoretic measures quantify how distinct the two types of distributions are from each other.

These statistical measures collectively provide a quantitative framework for assessing whether the parse score distributions can reliably distinguish between ambiguous and non-ambiguous sentences.

Chapter 5

Results

5.1 Prompt-Based Classification Results

Our baseline evaluation reveals significant limitations in LLMs’ capability to identify syntactic, lexical and coreferential ambiguity. Despite implementing chain-of-thought prompting strategies, the models demonstrated consistently low classification accuracy across different ambiguity categories. For Syntactic and Lexical classifications, most models performed below 50% accuracy, with only GPT-4 achieving higher accuracy rates (77.8% and 66.7%, respectively) when enhanced with the FlipFlop strategy(refer to Table 5.1). The prompts used in this study were designed following the strategies outlined in Section 4.2.1, with the complete set of prompts provided in Appendix A.1.

The performance analysis revealed distinct patterns across different ambiguity types and model architectures. Most notably, nearly all models failed to identify coreferential ambiguities, with only Phi-3 (128K) showing minimal success at 33.3% accuracy. Interestingly, larger parameter counts did not necessarily correlate with better performance; for instance, Llama3.1(8B) demonstrated superior Lexical classification compared to Llama3.1(405B). Generally, models showed stronger capability in syntactic ambiguity detection compared to lexical and coreferential categories.

While the FlipFlop strategy appeared to enhance performance, particularly for GPT-4, recent research by Laban et al. [34] suggests these improvements may primarily reflect the models’ sycophantic behavior rather than genuine disambiguation capabilities. This finding raises critical concerns about the reliability of current LLM-based approaches for ambiguity detection. Our results emphasize the necessity for developing more robust and dependable methodologies for identifying linguistic ambiguities, as current LLMs approaches prove inadequate for systematic ambiguity detection tasks.

Table 5.1: Model Accuracy for Syntactic, Lexical, and Coreferential Tests (in %)

Model	Syntactic (%)	Lexical (%)	Coreferential (%)
GPT-4	22.2	28.6	0.0
GPT-4 (FlipFlop experiment)	77.8	66.7	0.0
Llama3.1 8B	22.2	47.6	0.0
Llama3.1 405B	33.3	9.5	0.0
Phi-3 128 K	11.1	9.5	33.3

5.2 K-best Constituency Parsing for Syntactic Ambiguity Detection Results

5.2.1 Manual Verification of Parse Trees

PP Attachment Ambiguity

The initial analysis focused on validating whether the highest-scoring parse trees correspond to valid interpretations of ambiguous sentences. For the prepositional phrase attachment case “An enraged cow injured a farmer with an ax,” examination of the top-scoring parse trees revealed that two distinct interpretations were captured among the four highest-scoring trees. Specifically, the highest-scoring tree (score: 13.60) and the fourth highest-scoring tree (score: 12.31) represent the two valid interpretations of the sentence.

The structural difference lies in the attachment point of the prepositional phrase “with an ax”: in the highest-scoring tree, the PP is directly attached to the VP node, suggesting the cow used the ax, while in the fourth tree, an additional X node creates a different structural relationship that groups “farmer with an ax” together, corresponding to the interpretation where the farmer possessed the ax. Figure 5.4 illustrates these two distinct structural analyses.

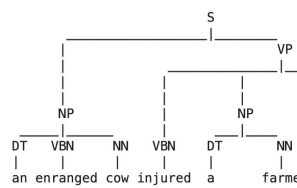


Figure 5.1: *

(a) Highest-scoring Tree: PP attached to VP

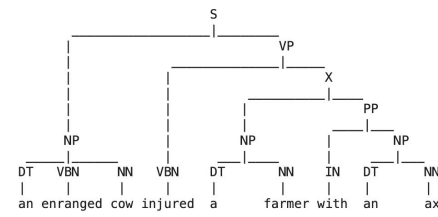


Figure 5.2: *

(b) Fourth-scoring Tree: PP attached to NP

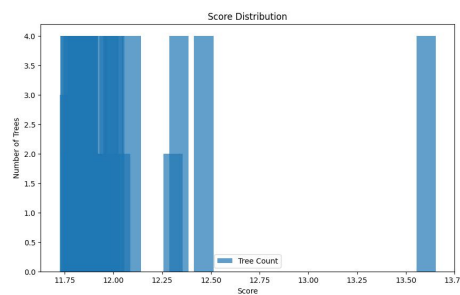


Figure 5.3: *

(c) Parse Tree Score Distribution

Figure 5.4: Illustration of two valid parse trees and their corresponding score distributions for the sentence “An enraged cow injured a farmer with an ax.” The structural difference lies in the attachment point of the PP “with an ax”.

Similarly, for another prepositional phrase attachment case “The professor said on Monday he would give an exam,” examination of the parse trees revealed that the two valid interpretations were captured among the four highest-scoring trees. The structural difference lies in the attachment of the temporal phrase “on Monday”: in one interpretation, it modifies the main verb “said” (indicating when the professor

made the statement), while in another tree, it modifies “would give” (specifying when the exam would be given).

This finding demonstrates that the k-best parsing approach can successfully capture multiple valid interpretations of PP-attachment ambiguity within its highest-scoring parse trees, though the scores may not always be consecutive in ranking.

Compound noun attachment ambiguity

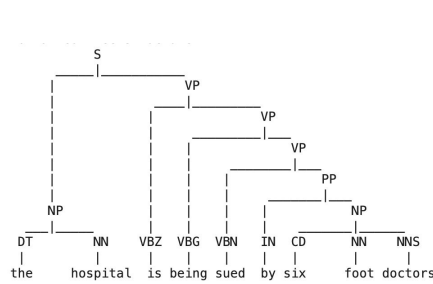


Figure 5.5: *

(a) Highest-scoring Tree: “six foot” as a compound noun modifying “doctors”

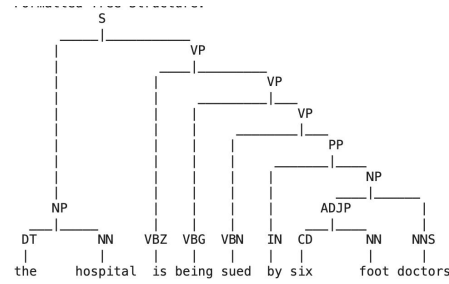


Figure 5.6: *

(b) Seventh-scoring Tree: “six” modifies “foot” as an adjective

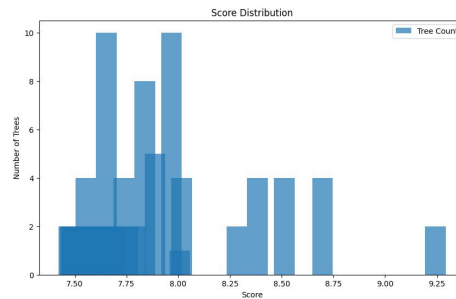


Figure 5.7: *

(c) Parse Tree Score Distribution

Figure 5.8: Illustration of two valid parse trees and their corresponding score distributions for the sentence “The hospital is being sued by six foot doctors.” The structural difference lies in the attachment of “six foot” as either a compound noun or an adjective phrase.

For compound noun attachment ambiguity, such as “The hospital is being sued by six foot doctors,” the desired interpretations were less consistently found among the highest-scoring trees. As illustrated in Figure 5.8, while the parse score distribution shows multiple high scores, the structural differences corresponding to the two valid interpretations (doctors who are six feet tall vs. doctors who specialize in foot medicine) are found in more widely separated trees – the highest-scoring tree (score: 9.24) and the seventh-ranked tree (score: 8.01). This contrasts with the PP attachment cases where the valid interpretations were typically found among the top-ranked trees. The more dispersed distribution and the need to look further down the ranking for valid interpretations suggests that the model finds compound noun attachment ambiguity more challenging to capture than PP attachment ambiguity. This difference in distribution patterns indicates that the model’s ability to capture ambiguity may depend on the specific type of syntactic structure involved.

5.2.2 Statistical Analysis of Parse Score Distributions

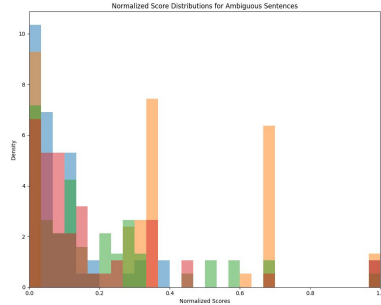


Figure 5.9: *
(a) Ambiguous Sentences

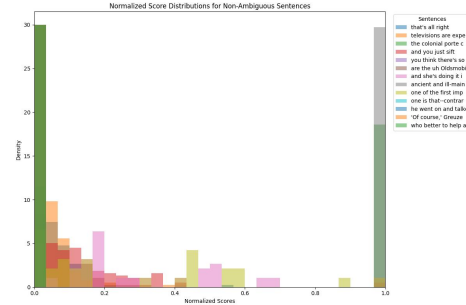


Figure 5.10: *
(b) Non-Ambiguous Sentences

Figure 5.11: Comparison of Normalized Score Distributions for Ambiguous and Non-Ambiguous Sentences.

These histograms depict the distributions of normalized k -best parse scores under our ambiguity-detection framework. Ambiguous sentences (left) typically exhibit multiple parse trees with comparably high scores. By contrast, non-ambiguous sentences (right) tend to have a single dominant parse tree with higher scores than all alternatives.

Three complementary statistical approaches were employed to analyze the parse score distributions:

Standard Deviation Analysis

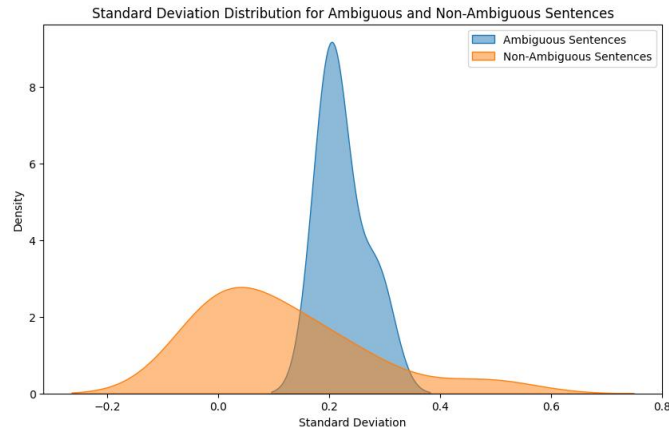


Figure 5.12: Comparison of Parse Score Variability Between Ambiguous and Non-Ambiguous Sentences

The standard deviation of parse scores was calculated to measure score dispersion. Statistical analysis revealed significant differences between ambiguous and non-ambiguous sentences, as shown in Figure 5.12. Ambiguous sentences showed a higher mean standard deviation (0.224 ± 0.036) compared to non-ambiguous sentences (0.116 ± 0.141), indicating more dispersed parse score distributions for ambiguous sentences. The variances of the two groups further highlight this

difference, with ambiguous sentences having a variance of 0.0013, while non-ambiguous sentences have a larger variance of 0.0200.

This difference was confirmed to be statistically significant through an independent t-test ($t = 2.357$, $p = 0.032$), indicating that the parse score distributions reliably differ between ambiguous and non-ambiguous sentences at the $p \leq 0.05$ level.

The higher standard deviation for ambiguous sentences aligns with the expectation that multiple valid interpretations would result in more dispersed parse scores, while non-ambiguous sentences would show more concentrated score distributions around a single interpretation.

Entropy Analysis

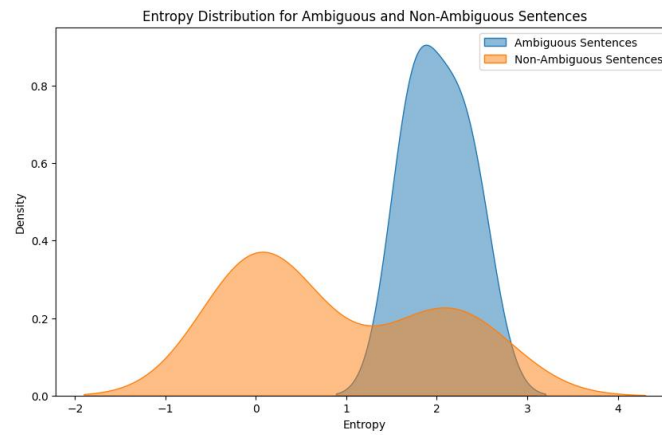


Figure 5.13: Parse Score Entropy Distribution for Ambiguous and Non-Ambiguous Sentences

Entropy calculations of normalized score distributions revealed distinct patterns between ambiguous and non-ambiguous sentences, as illustrated in Figure 5.13. Ambiguous sentences exhibited significantly higher mean entropy values (2.015 ± 0.296) compared to non-ambiguous sentences (0.871 ± 1.017), suggesting greater uncertainty in the parse score distributions for ambiguous sentences.

The difference in entropy distributions was found to be statistically significant through parametric testing ($t = 3.366$, $p = 0.004$), though the non-parametric Mann-Whitney U test showed a weaker significance ($U = 41.000$, $p = 0.093$). The higher entropy values for ambiguous sentences align with the theoretical expectation that multiple valid interpretations would result in more evenly distributed parse scores, while non-ambiguous sentences would show more concentrated distributions with lower entropy. This is visually evident in the entropy distribution plot, where ambiguous sentences consistently show higher entropy values clustered around 2.0-2.5, while non-ambiguous sentences display lower and more varied entropy values.

Distribution Divergence

The information-theoretic measures were employed to quantify the overall difference between ambiguous and non-ambiguous parse score distributions. The Jensen-Shannon divergence between the two distributions was 0.530, indicating a substantial difference in their probability distributions. This was further supported by the Kullback-Leibler divergence value of 11.708, which suggests a strong divergence

between the two distributions. These relatively high divergence values provide additional evidence that ambiguous and non-ambiguous sentences exhibit distinctly different parse score distribution patterns, supporting the effectiveness of using parse score distributions as a basis for ambiguity detection.

5.2.3 Performance Across Ambiguity Types

Analysis of different ambiguity categories revealed distinct patterns in parse score distributions. Our results demonstrate a clear distinction between ambiguous and non-ambiguous sentences, despite the presence of parsing errors in both categories. While the k-best parsing approach generates some incorrect trees for both types of sentences, this noise appears to be a systematic characteristic of the model rather than specific to either category.

Importantly, even with these parsing errors, the statistical analysis shows significant differences in the distribution patterns between ambiguous and non-ambiguous sentences. This finding is particularly meaningful because it suggests that the method's ability to distinguish ambiguous sentences is robust to general parsing errors. Since these errors occur consistently across both sentence types, the observed differences in score distributions can be attributed to the inherent structural ambiguity rather than model noise.

This robustness to parsing errors is a crucial feature of our approach, as it indicates that the method can reliably identify ambiguous sentences even in the presence of imperfect parsing. However, the generalizability of these findings may be limited. The test samples used in this study may contain structural patterns particularly suited for this method, raising questions about its applicability to a broader range of syntactic structures and sentence complexities. Further investigation is required to assess the method's performance in these scenarios.

Chapter 6

A Visual Resolution for Syntactic Ambiguity

6.1 Design Goals and Motivation

The visualization system is designed to make syntactic ambiguity immediately apparent and comprehensible to non-expert users. This system supports two distinct user scenarios: helping writers identify potential ambiguities in their text that can then be resolved through the addition of appropriate context, while also enabling readers to better understand multiple possible interpretations in both human-written text and machine-generated content from LLMs. While traditional parse tree representations are comprehensive, they typically require linguistic expertise to interpret, making them inaccessible to general users. Our approach focuses on making complex syntactic structures accessible through intuitive visual cues and explanations, bridging the gap between linguistic complexity and user understanding.

6.2 Visualization Components

To provide immediate awareness of potential ambiguity while maintaining natural reading flow, multiple visualization approaches are designed. When users open the interface, ambiguous phrases in the text are automatically highlighted, allowing for immediate recognition of potential ambiguity points. Users can then explore different visualization schemes, ranging from simple inline indicators to more detailed structural representations.

While traditional parse tree visualizations can be overwhelming for non-expert users, our approach builds upon these representations by selectively highlighting the differentiating subtrees to combat the counter intuitive nature of the tree. This focused visualization strategy helps users concentrate on the crucial structural differences that create ambiguity, rather than being overwhelmed by the complete syntactic structure. The inclusion of parse tree visualization, despite its technical nature, is justified by its unique ability to reveal the hierarchical relationships between sentence components that give rise to ambiguity.

The system implements four complementary visualization functions, each serving different aspects of ambiguity comprehension. This multi-faceted approach serves two primary purposes: it enables users to compare different representations and choose the most intuitive one for their understanding, while also providing a progressive learning path. Users can begin with simple, intuitive indicators and gradually transition to more sophisticated structural representations as their understanding of

syntactic relationships develops. This design philosophy acknowledges the varying levels of linguistic expertise among users while providing tools for deeper understanding of syntactic ambiguity.

Four visualization components are included, as described below. Throughout this chapter, we will use the following example sentence S to illustrate our visualization approaches: “An enraged cow injured a farmer with an ax.” We denote the key components as follows: w_1 = “injured” (main verb), w_2 = “a farmer” (object noun phrase), and w_3 = “with an ax” (prepositional phrase). The ambiguity in S arises from two possible interpretations: $[w_1, w_3]$ where w_3 modifies the action of injuring, and $[w_2, w_3]$ where w_3 describes the farmer. These complementary visualization components can be used either independently or in combination, providing users with flexible ways to understand and resolve syntactic ambiguity based on their preferences and needs.

Highlighting The design of highlighting is motivated by the fundamental nature of syntactic ambiguity, which typically arises from different possible attachments of sentence components. In sentence S , w_3 can be attached either to w_2 or to w_1 (describing the cow’s action). Color-based highlighting is employed to visualize these alternative groupings, making the structural differences immediately apparent to users without requiring linguistic expertise. In the interface implementation, detected ambiguous phrases are augmented with two interactive buttons positioned above the text. When users hover over the blue button, the system highlights one possible grouping in blue (e.g., $[w_1, w_3]$). Conversely, hovering over the orange button highlights an alternative grouping in orange (e.g., $[w_2, w_3]$). This interactive color-coding system provides an immediate visualization of how different word groupings lead to different interpretations of the sentence, as shown below in Fig. 6.1.

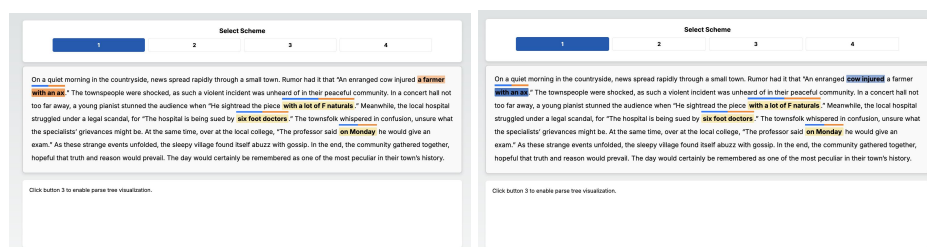


Figure 6.1: Highlighting

Contextual Explanations Building upon the concept as described in visual marking, contextual explanations provide explicit linguistic descriptions of the ambiguity present in the text. While visual markers highlight structural relationships through color, these explanations offer direct, natural language descriptions of how different interpretations arise. In the interface implementation, these explanations are triggered through user interaction. When users hover over an ambiguous phrase, a tooltip-style explanation appears above the text, as shown in Fig. 6.2. The explanation is automatically generated by analyzing the different parse trees produced by the system, translating the structural differences into clear, accessible language. For instance, when users hover over w_3 in sentence S , a tooltip appears stating: “There is an attachment ambiguity: w_3 could latch on to both w_1 and w_2 .”

This approach provides users with immediate insight into the nature of the ambiguity without requiring them to interpret complex syntactic structures themselves.

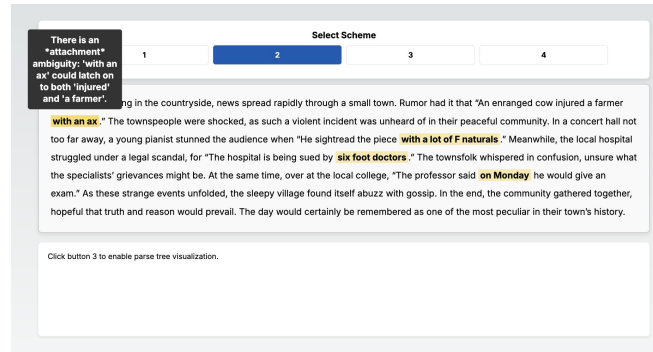


Figure 6.2: Contextual Explanations

The natural language explanations serve as an intuitive bridge between the visual highlighting and the underlying linguistic complexity, making syntactic ambiguity more accessible to users regardless of their linguistic background.

Disambiguation Examples Inspired by the disambiguation approach presented in Liu et al. [4], this feature provides explicit examples of how the ambiguous sentence could be rephrased to convey each intended meaning unambiguously. As shown in Fig. 6.3, in the interface implementation these examples appear in the same tooltip-style interface used for contextual explanations, providing a consistent user experience. When users hover over w_3 in sentence S , the system presents two unambiguous alternative phrasings: $S_1 =$ “The enraged cow used an ax to injure a farmer” (corresponding to $[w_1, w_3]$) and $S_2 =$ “The enraged cow injured a farmer who was carrying an ax” (corresponding to $[w_2, w_3]$). By expressing each interpretation through natural, complete sentences, this feature makes the different meanings immediately clear while remaining accessible to users without linguistic training.

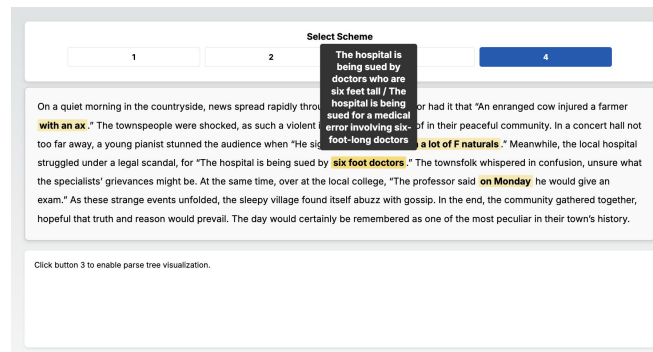


Figure 6.3: Disambiguation Examples: Providing explicit rephrasings to convey unambiguous interpretations.

Parsing Tree Visualization While parse trees are traditionally complex and potentially overwhelming for non-expert users, this feature implements a focused approach to tree visualization that emphasizes only the structurally different components. The design principle stems from the observation that syntactic ambiguity typically manifests as variations in specific subtrees while much of the overall structure remains identical. In the interface implementation, as shown in Fig. 6.4, parse trees are

displayed when users interact with ambiguous phrases, showing two alternative structural analyses side by side. For instance, when users hover over w_3 in sentence S , two parse trees appear, with highlighted regions showing the different attachment points—one where w_3 attaches to w_1 ($[w_1, w_3]$), and another where it modifies w_2 ($[w_2, w_3]$). This selective highlighting helps users focus on the crucial structural differences that give rise to ambiguity while avoiding the cognitive load of processing complete parse trees. The parse tree visualization, while more technical in nature, serves as a valuable complement to other visualization methods by explicitly showing the structural source of ambiguity.

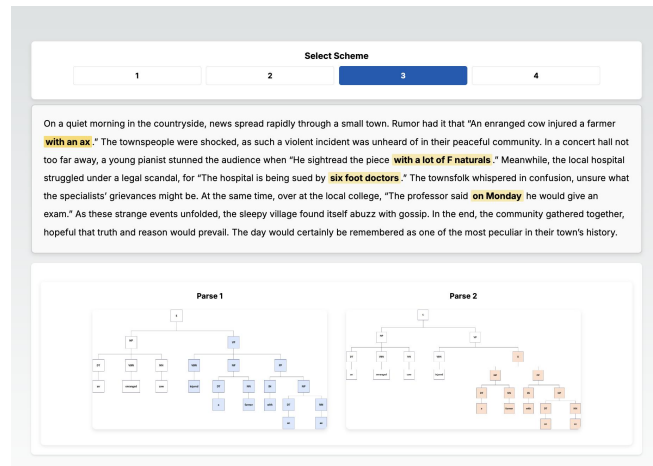


Figure 6.4: Parse Tree Visualization: Highlighting structural ambiguity through alternative parse tree representations.

6.3 Technical Implementation

The visualization system was implemented using React and Next.js, with styles defined through CSS modules to ensure a consistent visual presentation. The core functionality utilizes React's state management (via `useState` and `useEffect`) to handle interactive features such as text highlighting, tooltip displays, and visualization mode switches. Text processing is managed through a central rendering function that identifies highlightable phrases and generates styled segments. The interactive elements—highlight bars, tooltips, and parse tree displays—are implemented using CSS positioning and transitions, controlled by mouse event handlers.

Chapter 7

Discussion

7.1 Effectiveness of Different Approaches

The experimental results demonstrate that LLM prompt-based methods cannot reliably classify ambiguous sentences. This limitation suggests that despite their impressive performance in various NLP tasks, current LLMs may lack the sophisticated syntactic understanding required for consistent ambiguity detection.

The *k*-best parsing approach shows promising results, particularly in detecting specific types of syntactic ambiguity. The method performs notably well in detecting prepositional phrase-attachment ambiguity and compound noun attachment ambiguity, though with varying degrees of success. Notably, the system demonstrates better performance in identifying prepositional phrase-attachment cases, as evidenced by the higher likelihood of finding the two desired interpretations among the highest-scoring parse trees. While similar patterns are observed for compound noun attachment ambiguity, the desired interpretations are typically more difficult to identify from the score distributions. However, a comprehensive quantitative comparison of the model’s performance across these ambiguity types would require testing on larger, more diverse datasets. This performance variation suggests potential inherent biases in the parsing model, possibly stemming from differences in structural complexity and span size associated with different types of ambiguity.

7.2 Model Bias and Role of Ambiguity in Language

The varying performance across different ambiguity types raises interesting questions about the relationship between computational models and human language processing. The observed differences in detection difficulty may reflect inherent structural properties, such as span size, which could influence both model performance and human perception. This parallel between computational and human processing difficulties suggests a deeper connection between model behavior and cognitive processing of syntactic ambiguity.

Following Beck et al. [38], these findings support the view that ambiguity is not merely a problem to be solved but a fundamental feature of language that serves important functions. Ambiguity enables efficient communication by allowing shorter expressions while maintaining effectiveness through context. The analysis reveals that different interpretations often have natural preference rankings rather than equal status, which aligns with human intuition and usage patterns. This observation is particularly interesting as the correlation between parse scores and commonly preferred interpretations warrants further investigation - it might reveal insights into how both humans and models process ambiguous structures. These findings suggest

that the goal should not be to eliminate ambiguity but to understand and work with it effectively, particularly in the context of human-AI interaction.

7.3 Future Directions

Future work should focus on investigating the relationship between parse score distributions and human interpretation preferences. This could include developing more sophisticated metrics for quantifying ambiguity detection difficulty and exploring the integration of LLM capabilities for more nuanced ambiguity analysis. The observed correlation between parsing difficulty and ambiguity types also suggests potential avenues for improving parsing models through better understanding of structural complexity factors.

As for the interface, the current one while demonstrating the feasibility of making syntactic ambiguity accessible to non-expert users, could benefit from formal user studies to validate the effectiveness of different visualization approaches. Future developments could explore both technical and practical enhancements: the integration of LLM-generated visual explanations for more intuitive representations, and the adaptation of visualization methods for specific applications such as re-annotation tasks and writing assistance tools. In these practical scenarios, the system could help maintain annotation consistency or provide real-time feedback about potential ambiguities, though each application would require specific adaptations for platform integration and workflow considerations. Future work should focus on evaluating and refining these visualization methods in concrete application scenarios while exploring more accessible ways to bridge the gap between linguistic expertise and general user understanding.

This work opens up broader questions about human-AI interaction and the role of ambiguity in communication. Understanding how to leverage ambiguity constructively in human-AI communication could lead to more natural and effective interaction paradigms. Furthermore, incorporating natural language ambiguity into AI system design might help create more robust and human-like language processing systems. The challenge lies in finding ways to make complex syntactic structures accessible to non-expert users while maintaining the natural flexibility and efficiency that ambiguity provides in human language.

Chapter 8

Conclusion

This work presents a novel approach to syntactic ambiguity detection and visualization, addressing a critical challenge in human-AI interaction. Through experimental evaluation, we demonstrated that while LLM-based methods showed limitations in reliable ambiguity classification, the k-best parsing approach effectively detected different types of syntactic ambiguity, particularly in prepositional phrase attachment cases.

The statistical analysis of parse score distributions revealed significant differences between ambiguous and non-ambiguous sentences, validating our approach through multiple metrics including standard deviation, entropy, and distribution divergence measures. These findings suggest that parse score patterns can serve as reliable indicators of syntactic ambiguity.

Our visualization system makes these complex linguistic structures accessible to non-expert users, providing an interactive interface for exploring and understanding different interpretations. This work contributes not only to the technical aspects of ambiguity detection but also to the broader understanding of how ambiguity functions in language and human-AI communication.

The insights gained from this research suggest that future work should focus on investigating the relationship between computational parsing and human interpretation preferences, while also exploring practical applications in areas such as writing assistance and annotation tasks. This work represents a step toward more effective human-AI communication systems that can handle ambiguity in a more natural and intuitive way.

Bibliography

- [1] S. T. Piantadosi, H. Tily, and E. Gibson, “The communicative function of ambiguity in language,” *Cognition*, vol. 122, no. 3, pp. 280–291, 2012.
- [2] M. Y. Li, A. Liu, Z. Wu, and N. A. Smith, “A taxonomy of ambiguity types for nlp,” 2024.
- [3] M. J. Q. Zhang and E. Choi, “Clarify when necessary: Resolving ambiguity through interaction with lms,” 2023.
- [4] A. Liu, Z. Wu, J. Michael, A. Suhr, P. West, A. Koller, S. Swayamdipta, N. A. Smith, and Y. Choi, “We’re afraid language models aren’t modeling ambiguity,” 2023.
- [5] Y. Yuan, C. Malaviya, and M. Yatskar, “AmbiCoref: Evaluating human and model sensitivity to ambiguous coreference,” in *Findings of the Association for Computational Linguistics: EACL 2023*, A. Vlachos and I. Augenstein, Eds. Dubrovnik, Croatia: Association for Computational Linguistics, May 2023, pp. 1023–1030.
- [6] M. X. Liu, A. Sarkar, C. Negreanu, B. Zorn, J. Williams, N. Toronto, and A. D. Gordon, ““what it wants me to say”: Bridging the abstraction gap between end-user programmers and code-generating large language models,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’23. ACM, Apr. 2023, p. 1–31.
- [7] E. Pavlick and T. Kwiatkowski, “Inherent disagreements in human textual inferences,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 677–694, 2019.
- [8] Z. Wang, Y. Dong, O. Delalleau, J. Zeng, G. Shen, D. Egert, J. J. Zhang, M. N. Sreedhar, and O. Kuchaiev, “Helpsteer2: Open-source dataset for training top-performing reward models,” 2024.
- [9] R. Chan, A. Amini, and M. El-Assady, “Which spurious correlations impact reasoning in NLI models? a visual interactive diagnosis through data-constrained counterfactuals,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, D. Bollegala, R. Huang, and A. Ritter, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 463–470.
- [10] K. Nadh and C. Huyck, “Prepositional phrase attachment ambiguity resolution using semantic hierarchies,” in *9th IASTED International Conference on Artificial Intelligence and Applications*. 8168y, 2009.
- [11] A. M. Buckeridge and R. F. E. Sutcliffe, “Disambiguating noun compounds with latent semantic indexing,” in *COLING-02: COMPUTERM 2002: Second International Workshop on Computational Terminology*, 2002.

- [12] M. Lauer, “Conceptual association for compound noun analysis,” 1996.
- [13] A. Sennet, “Ambiguity,” 2023.
- [14] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 2024, draft of February 3, 2024.
- [15] Y. Tian, Y. Song, F. Xia, and T. Zhang, “Improving constituency parsing with span attention,” 2020.
- [16] N. Kitaev and D. Klein, “Constituency parsing with a self-attentive encoder,” 2018.
- [17] T. Kasami, “An efficient recognition and syntax-analysis algorithm for context-free languages,” 1965.
- [18] E. C. Ferstl, “Context effects in syntactic ambiguity resolution: The location of prepositional phrase attachment,” in *Proceedings of the Sixteenth Annual Conference of the Cognitive Science Society*. Routledge, 2019, pp. 295–300.
- [19] A. M. Buckeridge and R. F. Sutcliffe, “Disambiguating noun compounds with latent semantic indexing,” in *COLING-02: COMPUTERM 2002: Second International Workshop on Computational Terminology*, 2002.
- [20] M. Olteanu and D. Moldovan, “Pp-attachment disambiguation using large context,” in *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, 2005, pp. 273–280.
- [21] W. Liao, Z. Wang, K. Shum, A. B. Chan, and J. Hsiao, “Do large language models resolve semantic ambiguities in the same way as humans? the case of word segmentation in chinese sentence reading,” in *Proceedings of the Annual Meeting of the Cognitive Science Society*, vol. 46, 2024.
- [22] G. Kamath, S. Schuster, S. Vajjala, and S. Reddy, “Scope ambiguities in large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, p. 738–754, 2024.
- [23] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, “Clarifygpt: Empowering llm-based code generation with intention clarification,” 2023.
- [24] H. J. Kim, Y. Kim, C. Park, J. Kim, C. Park, K. M. Yoo, S. goo Lee, and T. Kim, “Aligning language models to explicitly handle ambiguity,” 2024.
- [25] OpenAI, “Introducing chatgpt,” 2022, accessed: 2024-07-26.
- [26] P. Jiang, J. Rayan, S. P. Dow, and H. Xia, “Graphologue: Exploring large language model responses with interactive diagrams,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST ’23. ACM, Oct. 2023.
- [27] F. Cheng, V. Zouhar, S. Arora, M. Sachan, H. Strobelt, and M. El-Assady, “RELIC: Investigating large language model responses using self-consistency,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, ser. CHI ’24. New York, NY, USA: Association for Computing Machinery, 2024.
- [28] R. Sánchez-Sáez, L. A. Leiva, J.-A. Sánchez, and J.-M. Benedí, “Interactive predictive parsing using a web-based architecture,” in *Proceedings of the NAACL HLT 2010 Demonstration Session*, 2010, pp. 37–40.

- [29] Y. Berzak, A. Barbu, D. Harari, B. Katz, and S. Ullman, “Do you see what i mean? visual resolution of linguistic ambiguities,” 2016.
- [30] A. Williams, N. Nangia, and S. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018, pp. 1112–1122.
- [31] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Transactions on Information Systems*, Nov. 2024.
- [32] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023.
- [33] P. Manakul, A. Liusie, and M. J. F. Gales, “Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models,” 2023.
- [34] P. Laban, L. Murakhovs’ka, C. Xiong, and C.-S. Wu, “Are you sure? challenging llms leads to performance drops in the flipflop experiment,” 2024.
- [35] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, and et al., “Gpt-4 technical report,” 2024.
- [36] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yearly, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla,

S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baeviski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024.

- [37] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacroce, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, “Phi-3 technical report: A highly capable language model locally on your phone,” 2024.
- [38] C. Beck, H. Booth, M. El-Assady, and M. Butt, “Representation problems in linguistic annotations: Ambiguity, variation, uncertainty, error and bias,” in *Proceedings of the 14th Linguistic Annotation Workshop*, S. Dipper and A. Zeldes, Eds. Barcelona, Spain: Association for Computational Linguistics, Dec. 2020, pp. 60–73.

Appendix A

Prompts and Model Descriptions

A.1 The Prompt Template

Below is the prompt template used in this study for ambiguity classification:
Given the following sentence, identify and classify the type of ambiguity present.
The ambiguity types to consider are:

1. Lexical Ambiguity: A word has multiple possible meanings.
2. Syntactic Ambiguity: The structure of a sentence allows for multiple interpretations.
3. Scopal Ambiguity: The scope of operators like quantifiers or negations can vary.
4. Elliptical Ambiguity: Ellipsis in a sentence leads to unclear references.
5. Collective/Distributive Ambiguity: A plural expression can refer to individuals or a group.
6. Implicative Ambiguity: A sentence implies something that isn't explicitly stated.
7. Presuppositional Ambiguity: The background assumption or presupposition in a sentence is unclear.
8. Idiomatic Ambiguity: A phrase can be interpreted literally or as an idiom.
9. Coreferential Ambiguity: It's unclear which noun a pronoun or phrase refers to.
10. Generic/Non-Generic Ambiguity: A statement can refer to a general category or a specific instance.
11. Type/Token Ambiguity: A term can refer to a general category (type) or a specific instance (token).

Please think step by step and provide a brief explanation for your classification.
Sentence: [input sentence]

This prompt was designed to guide the language model in systematically analyzing and classifying different types of ambiguity in given sentences. The prompt includes clear definitions for each ambiguity type to ensure consistent classification criteria.

